

BANK ACCOUNT MANAGEMENT SYSTEM

Experiment No: 02

Date: 15/09/2026

Title: Bank Account Management System using Classes and Objects

Objective / Aim

To design and implement a Bank Account Management System in Java using classes and objects, demonstrating object-oriented programming concepts such as attributes, constructors, object creation, inheritance, method overriding, deposits, withdrawals, interest calculation, fund transfers, and account detail display.

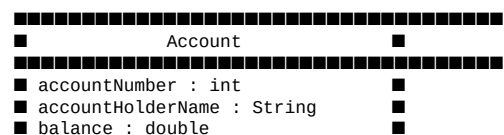
System Requirements

Hardware	Software
Standard PC/Laptop	Windows / Linux / macOS
Intel Core i3/i5 or equivalent	JDK 17 or JDK 21
Minimum 4 GB RAM	Java compiler (javac) and runtime (java)
Minimum 500 MB free storage	Eclipse / IntelliJ IDEA / NetBeans / VS Code

Algorithm / Logical Description

1. Create the Account base class with account number, holder name, balance, and account type.
2. Create an Account constructor to initialize the account details.
3. Implement deposit() to add money to the balance.
4. Implement withdraw() to check balance and subtract the withdrawal amount.
5. Implement transfer() to subtract money from the sender and add it to the receiver.
6. Implement displayAccountDetails() to show account information.
7. Create SavingsAccount by inheriting Account and add interestRate.
8. Use super() to initialize inherited Account values and set the type to Savings.
9. Implement calculateInterest() and add the calculated interest to the balance.
10. Create CurrentAccount by inheriting Account and add overdraftLimit.
11. Override withdraw() so CurrentAccount can use the overdraft limit.
12. Create SavingsAccount and CurrentAccount objects in main().
13. Perform deposits, withdrawals, interest calculation, and fund transfer.
14. Display the final account details and final balances.

UML Class Diagram



```

    ■ accountType : String
    ■ deposit(amount)
    ■ withdraw(amount)
    ■ transfer(receiver, amount)
    ■ displayAccountDetails()
    ■
    ■
    ■
    ■ SavingsAccount
    ■ CurrentAccount
    ■ interestRate
    ■ overdraftLimit
    ■ calculateInterest
    ■ withdraw(amount)
    ■
    ■
    ■ BankAccountDemo
    ■
    ■ main(String[] args)
    ■

```

Source Code

```

// Base class
class Account {

    // Attributes
    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;

    // Constructor to initialize account details
    Account(int accountNumber, String accountHolderName,
            double balance, String accountType) {

        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
        this.accountType = accountType;
    }

    // Method to deposit money
    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
    }

    // Method to withdraw money
    void withdraw(double amount) {

        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }

    // Method to transfer money
    void transfer(Account receiver, double amount) {

        if (amount <= balance) {
            balance = balance - amount;
            receiver.balance = receiver.balance + amount;

            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Insufficient balance for transfer");
        }
    }

    // Method to display account details
    void displayAccountDetails() {

        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolderName);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}

// SavingsAccount inherits Account
class SavingsAccount extends Account {

    // Additional attribute

```

```

double interestRate;

// Constructor
SavingsAccount(int accountNumber, String accountHolderName,
               double balance, double interestRate) {

    super(accountNumber, accountHolderName, balance, "Savings");
    this.interestRate = interestRate;
}

// Method to calculate interest
void calculateInterest() {

    double interest = balance * interestRate / 100;

    balance = balance + interest;

    System.out.println("Interest Added: " + interest);
}
}

// CurrentAccount inherits Account
class CurrentAccount extends Account {

    // Additional attribute
    double overdraftLimit;

    // Constructor
    CurrentAccount(int accountNumber, String accountHolderName,
                  double balance, double overdraftLimit) {

        super(accountNumber, accountHolderName, balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Overriding withdraw() method
    @Override
    void withdraw(double amount) {

        if (amount <= balance + overdraftLimit) {

            balance = balance - amount;

            System.out.println("Withdrawn: " + amount);

        } else {

            System.out.println("Withdrawal exceeds overdraft limit");
        }
    }
}

// Main class
class BankAccountDemo {

    public static void main(String[] args) {

        // Creating SavingsAccount object
        SavingsAccount savings =
            new SavingsAccount(101, "Rahul", 10000, 5);

        // Creating CurrentAccount object
        CurrentAccount current =
            new CurrentAccount(102, "Priya", 5000, 3000);

        // Display details before transactions
        System.out.println("==== BEFORE TRANSACTIONS =====");

        System.out.println("\nSavings Account Details:");
        savings.displayAccountDetails();

        System.out.println("\nCurrent Account Details:");
        current.displayAccountDetails();

        // Deposit operation
        System.out.println("\n===== DEPOSIT =====");

        savings.deposit(2000);
        current.deposit(3000);

        // Withdrawal operation
        System.out.println("\n===== WITHDRAWAL =====");

        savings.withdraw(1000);
        current.withdraw(9000);
        // Interest calculation
    }
}

```

```

System.out.println("\n===== INTEREST CALCULATION =====");

savings.calculateInterest();

// Fund transfer
System.out.println("\n===== FUND TRANSFER =====");

savings.transfer(current, 2000);

// Display details after transactions
System.out.println("\n===== AFTER TRANSACTIONS =====");

System.out.println("\nSavings Account Details:");
savings.displayAccountDetails();

System.out.println("\nCurrent Account Details:");
current.displayAccountDetails();

// Final balances
System.out.println("\n===== FINAL BALANCES =====");

System.out.println("Savings Account Final Balance: "
    + savings.balance);

System.out.println("Current Account Final Balance: "
    + current.balance);
}
}

```

Completion and Execution Commands

1. Save the program as BankAccountDemo.java.
2. Open Command Prompt / Terminal and move to the folder containing the file.
3. Compile the program:

```
javac BankAccountDemo.java
```

4. Execute the program:

```
java BankAccountDemo
```

5. If there are no compilation errors, the program executes and displays the banking transactions and final balances.

Output Screenshot

The console output below was generated by compiling and executing the supplied source code.

```
===== BEFORE TRANSACTIONS =====

Savings Account Details:
Account Number: 101
Account Holder: Rahul
Account Type: Savings
Balance: 10000.0

Current Account Details:
Account Number: 102
Account Holder: Priya
Account Type: Current
Balance: 5000.0

===== DEPOSIT =====
Deposited: 2000.0
Deposited: 3000.0

===== WITHDRAWAL =====
Withdrawn: 1000.0
Withdrawn: 9000.0

===== INTEREST CALCULATION =====
Interest Added: 550.0

===== FUND TRANSFER =====
Transferred: 2000.0

===== AFTER TRANSACTIONS =====

Savings Account Details:
Account Number: 101
Account Holder: Rahul
Account Type: Savings
Balance: 9550.0

Current Account Details:
Account Number: 102
Account Holder: Priya
Account Type: Current
Balance: 1000.0

===== FINAL BALANCES =====
Savings Account Final Balance: 9550.0
Current Account Final Balance: 1000.0
```

Result / Completion

The Bank Account Management System was successfully implemented and executed in Java. It demonstrates classes and objects, constructors, inheritance, method overriding, deposit, withdrawal, interest calculation, fund transfer, and account detail display.